Universidad de Alicante

Escuela Politécnica Superior

Grado en Ingeniería en Inteligencia Artificial | Curso 4º (2026/2027)

Robótica Inteligente (33683)

Memoria Técnica de Práctica 1

Introducción a las Herramientas Básicas para Robótica Móvil y Manipulación: ROS 2, Gazebo Harmonic, Nav2, MoveIt 2 y Oxford Spires Dataset

---

Profesor / Tutor:

M.A. Muñoz-Bañón
Dpto. de Física, Ingeniería de Sistemas
y Teoría de la Señal

Equipo de Prácticas:

Santiago Álvarez González (`santvz6`)

Fecha de Evaluación:

Octubre de 2026

Entorno de ejecución: ROS 2 Jazzy Jalisco | Gazebo Harmonic 8.0 | Ubuntu 24.04 LTS (Docker)

Índice

1. Introducción y Estructura Global de la Práctica **1**
1.1. Estructura Modular de la Práctica . . . 1

2. Bloque 2 – Mapeo y Navegación Autónoma con TurtleBot 4 y Nav2 **1**
2.1. Contexto y Plataforma Móvil Diferencial . . . 1
2.2. Mapeo del Entorno con SLAM y Teleoperación . . . 2
2.2.1. Despliegue y Exploración Manual . . . 2
2.3. Localización Basada en Mapa con Filtro de Partículas (AMCL) . . . 2
2.4. Pila de Navegación Nav2 y Planificación de Trayectorias . . . 3
2.4.1. Mapas de Coste (Costmaps 2D) . . . 3
2.4.2. Planificador Global y Controlador Local . . . 4
2.5. Introspección del Middleware ROS 2 (Sección 6.1) . . . 4
2.5.1. Mapeo Detallado de Nodos e Interfaces . . . 4
2.5.2. Grafo Computacional Dinámico (`rqt_graph`) . . . 4
2.6. Explicación del Flujo de Información en Navegación Autónoma . . . 6
2.7. Análisis e Interpretación del Árbol de Transformaciones (TF2) . . . 6
2.8. Tarea de Programación Opcional: Control Autónomo en Python (Sección 6.2) . . . 6
2.8.1. Aspectos Técnicos de la Implementación . . . 7
2.8.2. Validación Experimental . . . 7

3. Bloque 4 – Manipulación Robótica (UR3e y MoveIt 2) **7**
3.1. Contexto y Arquitectura de los Tres Pilares . . . 7
3.2. Despliegue y Control Interactivo en RViz2 . . . 8
3.2.1. Operación en el Espacio de Trabajo . . . 8
3.3. Introspección del Middleware ROS 2 (Sección 6.1) . . . 8
3.3.1. Mapeo Detallado de Nodos e Interfaces . . . 9
3.3.2. Grafo Computacional Dinámico (`rqt_graph`) . . . 9
3.4. Explicación del Flujo de Información y Cinemática . . . 9
3.5. Análisis e Interpretación del Árbol de Transformaciones (TF2) . . . 10
3.6. Tarea de Programación Opcional: Control en Python (Sección 6.2) . . . 11

1 Introducción y Estructura Global de la Práctica

La presente memoria técnica documenta el desarrollo integral y el análisis exhaustivo de la **Práctica 1 de Robótica Inteligente**. El objetivo central consiste en capacitar al alumnado en las herramientas de vanguardia para investigación e industria en robótica: el middleware distribuido **ROS 2 (Jazzy Jalisco)**, el simulador dinámico **Gazebo Harmonic** y los frameworks avanzados de navegación autónoma (**Nav2**), odometría LiDAR-inercial (**RKO-LIO**) y manipulación robótica industrial (**MoveIt 2**).

Toda la suite de desarrollo opera encapsulada sobre un contenedor persistente de **Docker** con aceleración gráfica por hardware (GPU NVIDIA dedicada, soporte PRIME Offload y paso directo X11), eliminando conflictos de dependencias y asegurando la reproducibilidad íntegra de los experimentos.

1.1 Estructura Modular de la Práctica

De acuerdo con las directrices de la guía docente, la práctica se estructura en cuatro bloques secuenciales:

- **Bloque 1 – Fundamentos de ROS 2:** Dominio de la arquitectura orientada a grafos de nodos, canales asíncronos (tópicos), llamadas síncronas (servicios) y tareas continuas con retroalimentación (acciones).
- **Bloque 2 – Mapeo y Navegación Autónoma (Nav2):** Generación de mapas de ocupación 2D mediante SLAM, localización probabilística con filtros de partículas (AMCL) y control de plataforma móvil diferencial *TurtleBot 4* empleando la pila *Nav2*.
- **Bloque 3 – Odometría LiDAR-Inercial (LIO) con Datos Reales:** Procesamiento sensorial denso sobre secuencias reales del *Oxford Spires Dataset* (Keble College) mediante el pipeline *RKO-LIO*. *[Desarrollo documentado en anexo correspondiente]*.
- **Bloque 4 – Manipulación Robótica (MoveIt 2):** Control cinemático cartesiano y articular del brazo industrial *Universal Robots UR3e* (6 DOF) en simulación dinámica y planificación libre de colisiones.

2 Bloque 2 – Mapeo y Navegación Autónoma con TurtleBot 4 y Nav2

2.1 Contexto y Plataforma Móvil Diferencial

En este bloque abordamos el problema clásico de la robótica móvil: cómo un robot autónomo puede explorar un entorno desconocido, construir un mapa digital de ocupación, ubicarse con precisión dentro de él y desplazarse de un punto a otro de forma completamente autónoma sorteando obstáculos estáticos y dinámicos.

La plataforma utilizada es el **TurtleBot 4**, un robot móvil de tracción diferencial estándar en investigación educativa que integra:

- **Base motriz diferencial (iRobot Create 3):** Dos ruedas motrices independientes y dos ruedas locas de apoyo, equipada con encoders ópticos en cada rueda para el cálculo de la odometría cinemática y sensores de contacto (*bumpers*).
- **Sensor LiDAR 2D (RPLIDAR):** Escáner láser montado en la parte superior del chasis que proporciona barridos de 360 grados en el plano horizontal a través del tópico `/scan`, permitiendo detectar paredes y obstáculos en un radio de hasta 12 metros.
- **Cámara RGB-D estéreo (OAK-D-PRO):** Módulo de visión espacial para percepción de profundidad tridimensional.

Todo el sistema opera simulado en **Gazebo Harmonic**, sincronizado mediante el reloj de simulación de ROS 2 (`/clock`) y coordinado a través del middleware ROS 2 Jazzy.

2.2 Mapeo del Entorno con SLAM y Teleoperación

Para que un robot navegue de forma global, necesita conocer previamente la geometría del espacio. Dado que partimos de un entorno desconocido, recurrimos al paradigma **SLAM** (*Simultaneous Localization and Mapping*), donde el robot estima simultáneamente su propia trayectoria y construye el mapa del entorno.

2.2.1 Despliegue y Exploración Manual

El proceso de mapeo se estructuró en los siguientes pasos:

1. **Arranque del simulador:** Se cargó el mundo virtual de laboratorio con el TurtleBot 4 mediante:

```
ros2 launch turtlebot4_gz_bringup turtlebot4_gz.launch.py model:=standard rviz2
    :=false
```

2. **Lanzamiento del algoritmo SLAM:** Se ejecutó el nodo de mapeo (`slam_toolbox` / Cartographer):

```
ros2 launch turtlebot4_navigation slam.launch.py
```

3. **Teleoperación por teclado:** Para cubrir todas las estanterías y pasillos del laboratorio, se utilizó el nodo `teleop_twist_keyboard`, enviando comandos de velocidad angular y lineal (`TwistStamped`) a través de la interfaz de control:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r /cmd_vel:=/
    cmd_vel_unstamped
```

4. **Almacenamiento del mapa:** Una vez recorrido todo el entorno y cerrado el lazo del mapa en RViz2, se exportó la rejilla de ocupación 2D generada mediante la herramienta `nav2_map_server`:

```
ros2 run nav2_map_server map_saver_cli -f /home/developer/ros2_ws/
    mapa_laboratorio
```

   Este comando produjo dos ficheros: `mapa_laboratorio.pgm` (imagen rasterizada en escala de grises donde el negro representa obstáculos, el blanco espacio libre y el gris celdas desconocidas) y `mapa_laboratorio.yaml` (metadatos con resolución de 0.03 m/celda y punto de origen).

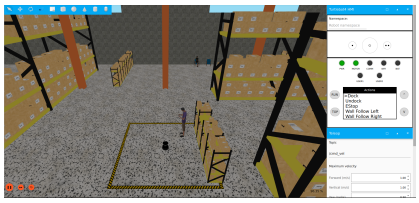
2.3 Localización Basada en Mapa con Filtro de Partículas (AMCL)

Una vez generado el mapa estático, el robot ya no necesita calcular SLAM (proceso computacionalmente costoso), sino que pasa a utilizar el algoritmo **AMCL** (*Adaptive Monte Carlo Localization*).

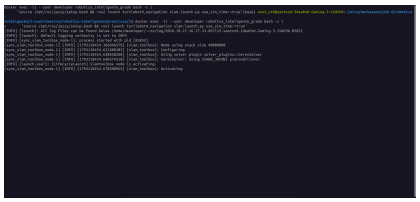
AMCL es un filtro de partículas que estima la posición y orientación del robot (x, y, θ) en el mapa comparando en tiempo real las lecturas del escáner láser (`/scan`) con las paredes y obstáculos de la rejilla de ocupación:

1. **Carga del mapa (`map_server`):** El nodo `map_server` publica el mapa estático en el tópico `/map` con durabilidad *transient local*.

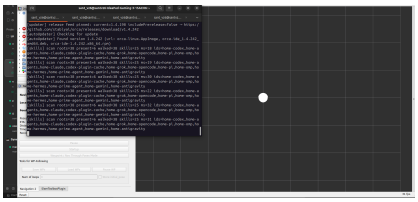
2. **Pose Inicial (*2D Pose Estimate*):** Dado que el filtro de partículas necesita una hipótesis de partida, se proporciona una estimación aproximada mediante la herramienta interactiva de RViz2 (o publicando en el tópico `/initialpose`).



(a) TurtleBot 4 en Gazebo Harmonic.



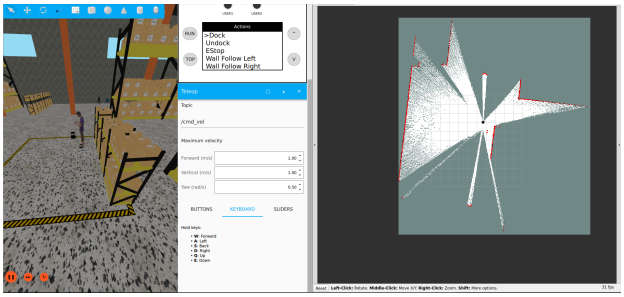
(b) Lanzamiento del nodo SLAM.



(c) Construcción del mapa en RViz2.



(d) Teleoperación manual con teclado.

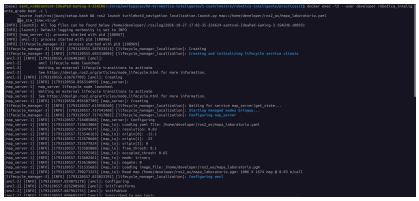


(e) Inspección combinada Gazebo + RViz2 durante el mapeo.

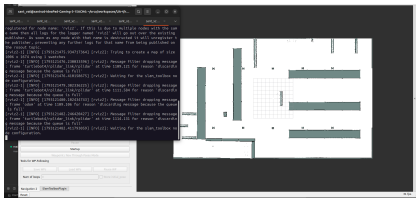
Figura 1: Secuencia experimental completa de la fase de Mapeo (SLAM): despliegue en simulación, exploración guiada por teleoperación y generación de la rejilla de ocupación 2D.

3. **Convergencia de partículas:** AMCL genera una nube de hipótesis representadas mediante flechas verdes (`/particle_cloud`). Conforme el robot recibe nuevas lecturas LiDAR y se desplaza, las partículas que coinciden con la geometría del entorno aumentan su peso probabilístico y las demás desaparecen, haciendo que la nube colapse sobre la posición real del vehículo.

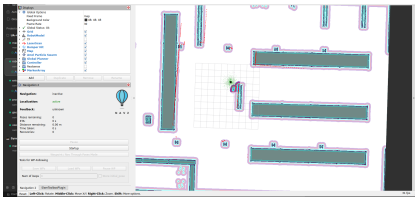
4. **Corrección de la odometría:** Con la pose estimada, AMCL publica continuamente la transformación dinámica `map ->odom`, corrigiendo en tiempo real la deriva acumulada por el deslizamiento de las ruedas en la odometría.



(a) Lanzamiento de `map_server` y AMCL.



(b) Mapa estático cargado en RViz2.



(c) Pose estimada y capas de *costmap*.

Figura 2: Fase de localización probabilística: servicio de mapas, inyección de pose inicial en RViz2 y activación de las capas de coste global.

2.4 Pila de Navegación Nav2 y Planificación de Trayectorias

Con el robot correctamente localizado en el mapa, se activa la pila de navegación autónoma **Nav2** (`nav2.launch.py`). Nav2 se estructura en una arquitectura modular gobernada por un Árbol de Comportamiento (*Behavior Tree*):

2.4.1 Mapas de Coste (Costmaps 2D)

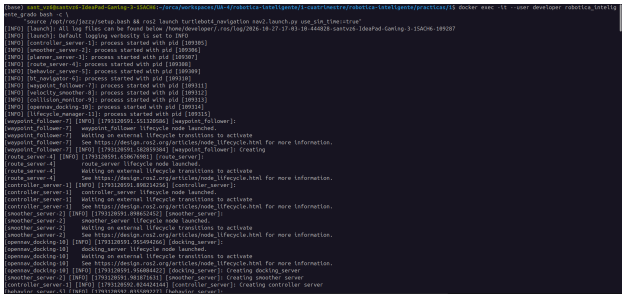
Para navegar con seguridad, Nav2 mantiene dos representaciones espaciales basadas en celdas de coste (donde 0 representa espacio libre, 254 obstáculo letal y los valores intermedios zonas de precaución):

- **Global Costmap:** Abarca todo el mapa del laboratorio. Integra la capa estática del mapa (`static_layer`), la capa de obstáculos de sensores (`obstacle_layer`) y una capa de inflado (`inflation_layer`) que expande los límites de los obstáculos en función del radio del chasis del robot, evitando que el planificador trace rutas rozando paredes.
- **Local Costmap:** Una ventana móvil de dimensiones reducidas centrada en el robot. Se actualiza a alta frecuencia con los datos frescos del LiDAR para detectar obstáculos imprevistos o personas en movimiento que no aparecían en el mapa original.

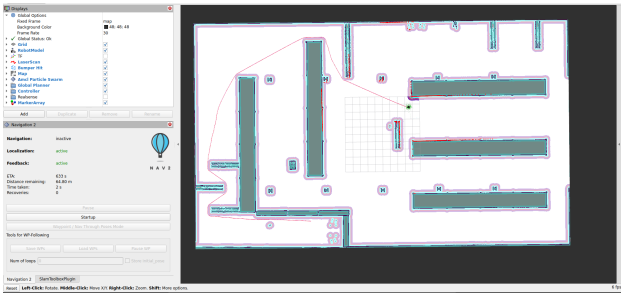
2.4.2 Planificador Global y Controlador Local

La navegación se divide en dos niveles complementarios:

1. **Planificador Global (`planner_server`):** Emplea el plugin *Navfn* (basado en el algoritmo A^* / Dijkstra) sobre el *Global Costmap* para calcular el camino geométrico óptimo desde la posición actual hasta el objetivo fijado.
2. **Controlador Local (`controller_server`):** Ejecuta el algoritmo *MPPI* (*Model Predictive Path Integral*) sobre el *Local Costmap*. Genera miles de trayectorias candidatas en tiempo real respetando la cinemática diferencial del robot y selecciona la velocidad lineal y angular óptima, esquivando obstáculos imprevistos y publicando consignas en `/cmd_vel`.
3. **Navegador de Comportamientos (`bt_navigator`):** Supervisa la ejecución global. Si el robot se queda atascado o un obstáculo bloquea el paso, activa el servidor de recuperaciones (`behavior_server`) para retroceder (*backup*) o girar sobre su eje (*spin*) antes de replanificar.



(a) Inicialización de servidores Nav2.



(b) Navegación hacia objetivo interactivo (*2D Goal Pose*).

Figura 3: Activación de la pila Nav2 y desplazamiento autónomo hacia una meta seleccionada interactivamente en RViz2.

2.5 Introspección del Middleware ROS 2 (Sección 6.1)

Siguiendo las indicaciones de la Sección 6.1 del guion, se procedió a analizar en detalle la topología de nodos, tópicos y servicios que sustentan la navegación del TurtleBot 4.

2.5.1 Mapeo Detallado de Nodos e Interfaces

La Tabla 1 resume las entradas, salidas, servicios y acciones de los nodos centrales del sistema de navegación.

2.5.2 Grafo Computacional Dinámico (`rqt_graph`)

La inspección visual del grafo de computación se obtuvo mediante `rqt_graph` (Figura 4). En ella se aprecia con claridad la separación entre la capa de estimación sensorial (AMCL procesando escaneos láser para publicar TF), la capa de decisión (Nav2 planificando y controlando) y el actuador diferencial final gobernado por `diffdrive_controller`.

Nodo Activo	Tópicos Suscritos	Tópicos Publicados	Servicios / Acciones
`/map_server`	Parámetros del ciclo de vida	`/map`, `/map_metadata`	Srv: `/map_server/load_map`, `/map_server/change_state`
`/amcl`	`/scan`, `/tf`, `/initialpose`, `/map`	`/amcl_pose`, `/particle_cloud`, `/tf` (`map->odom`)	Srv: `/amcl/set_initial_pose`, gestión ciclo de vida
`/planner_server`	`/global_costmap/costmap`, `/tf`	`/plan`, `/global_costmap/published_footprint`	Acción: `/compute_path_to_pose`. Srv: `/is_path_valid`
`/controller_server`	`/local_costmap/costmap`, `/odom`, `/tf`	`/cmd_vel_nav`, marcadores trayectoria (`/optimal_trajectory`)	Acción: `/follow_path`
`/bt_navigator`	`/goal_pose`, `/tf`, feedback de acciones	`/transformed_global_plan`	Acciones: `/navigate_to_pose`, `/navigate_through_poses`
`/behavior_server`	`/tf`, `/local_costmap/costmap`	Comandos de recuperación	Acciones: `/spin`, `/backup`, `/drive_on_heading`, `/wait`
`/diffdrive_controller`	`/diffdrive_controller/cmd_vel`, `/clock`	`/odom`, `/tf` (`odom->base_link`)	Srv: Control de actuadores en tiempo real

Tabla 1: Mapeo de interfaces de los componentes clave de la pila de navegación (Bloque 2).

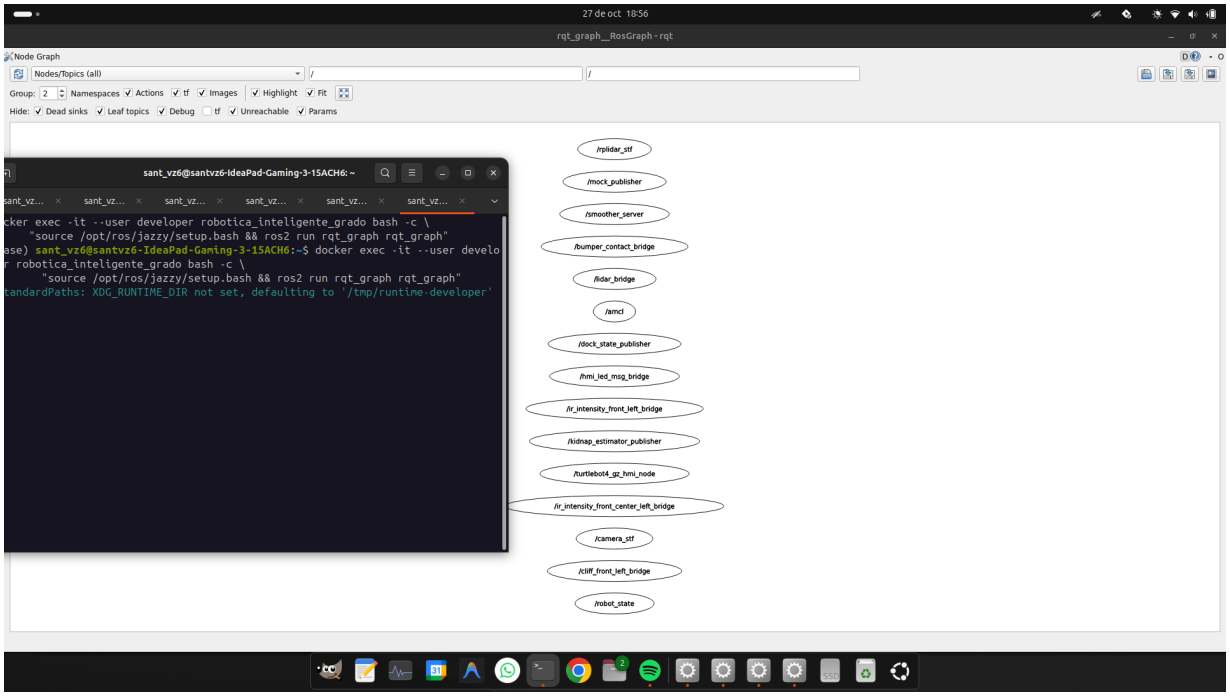


Figura 4: Grafo dinámico de nodos y canales de comunicación de la navegación autónoma capturado mediante `rqt_graph`.

2.6 Explicación del Flujo de Información en Navegación Autónoma

Para entender cómo se articula la toma de decisiones del robot durante una misión, analizamos el recorrido secuencial de la información a través de los nodos:

1. **Adquisición sensorial:** Los encoders de las ruedas miden el giro de cada motor y publican la odometría en `/odom`. En paralelo, el sensor RPLIDAR realiza barridos a 10 Hz publicando las distancias medidas en `/scan`.

2. **Localización en el mapa:** El nodo `amcl` recibe las lecturas de `/scan` y `/odom`, y las contrasta con el mapa estático (`/map`). A partir de esta correlación, calcula el error de deriva y emite la corrección en el marco `map ->odom`.

3. **Recepción del objetivo:** El usuario o un programa cliente envía una coordenada destino al tópico `/goal_pose` o mediante una acción a `bt_navigator`.

4. **Planificación global del camino:** El nodo `planner_server` consulta el *Global Costmap* y calcula una ruta libre de paredes que conecta la posición actual del robot con la meta.

5. **Control local y seguimiento reactivo:** El nodo `controller_server` recibe la ruta global y, consultando el *Local Costmap* (que incluye obstáculos en tiempo real), calcula las velocidades lineal (v_x) y angular (ω_z) requeridas.

6. **Filtrado y seguridad:** La velocidad pasa por el nodo `velocity_smoother` (para evitar tirones y respetar límites de aceleración) y por el monitor de colisión (`collision_monitor`), antes de llegar al controlador de las ruedas (`diffdrive_controller`) que mueve físicamente al TurtleBot 4 en el simulador.

2.7 Análisis e Interpretación del Árbol de Transformaciones (TF2)

La coherencia espacial de un robot móvil se gestiona mediante el árbol de transformaciones **TF2**, verificado mediante `ros2 run tf2_tools view_frames`. En robótica móvil existen tres marcos de referencia estándar:

$$\texttt{map} \xrightarrow{\text{AMCL (corrección)}} \texttt{odom} \xrightarrow{\text{encoders (cinemática)}} \texttt{base_link} \longrightarrow \{\texttt{rplidar_link}, \texttt{camera_link}, \dots\}$$

- `map ->odom` **(Transformación dinámica no continua):** Calculada y publicada por **AMCL**. Representa la corrección absoluta en el mapa global. Dado que las ruedas pueden patinar sobre el suelo, la odometría pura acumula error con el tiempo; AMCL ajusta este enlace de forma continua para que el robot nunca se desvíe en el mapa.

- `odom ->base_link` **(Transformación dinámica continua y de alta frecuencia):** Publicada a 50 Hz por el controlador de tracción diferencial. Es una estimación suave y sin saltos basada en la integración del giro de las ruedas.

- **Transformaciones estáticas de la plataforma:** Publicadas en `/tf_static` por el nodo `robot_state_publisher`. Representan la traslación y orientación física constante desde el marco base `base_link` hacia cada componente sensor (LiDAR, cámara, chasis y ruedas).

2.8 Tarea de Programación Opcional: Control Autónomo en Python (Sección 6.2)

Cumpliendo con la Sección 6.2 del guion de la práctica, se implementó el script en Python `nav2_send_goal.py` alojado en el espacio de trabajo persistente (`~/ros2_ws/p1_scripts/`). Este script prescinde de clics manuales en RViz2, conectándose como un cliente nativo a la acción de ROS 2 `NavigateToPose` ofrecida por `bt_navigator`.

2.8.1 Aspectos Técnicos de la Implementación

- **Sincronización con el reloj simulado (`use_sim_time`):** Para operar de forma robusta en Gazebo sin desincronizaciones temporales con TF2, el nodo inicializa el parámetro `use_sim_time=True` y espera a recibir la primera señal de `/clock` antes de sellar el mensaje `PoseStamped` de la meta.
- **Conversión de Orientación:** El script recibe el ángulo de orientación deseado en grados sexagesimales (`yaw`) y lo convierte matemáticamente en un cuaternión unitario $\mathbf{q} = (x, y, z, w)$ compatible con los estándares de ROS 2.
- **Retroalimentación Asíncrona en Tiempo Real:** El cliente escucha las llamadas de realimentación (*feedback callback*) de Nav2, imprimiendo en consola la distancia euclídea restante hasta la meta, el tiempo transcurrido de misión y el número de maniobras de recuperación ejecutadas.

2.8.2 Validación Experimental

Se ejecutó el envío de una meta situada en la zona central despejada del almacén ($X = 1{,}50\,\text{m}, Y = -0{,}50\,\text{m}, \text{Yaw} = 90°$):

```
python3 nav2_send_goal.py --x 1.5 --y -0.5 --yaw 90
```

El planificador global de Nav2 aceptó el objetivo de forma inmediata y trazó la trayectoria libre de obstáculos. Durante el desplazamiento, el cliente de acción monitorizó en tiempo real la aproximación continua del vehículo móvil mediante las llamadas de retroalimentación asíncrona, observándose la reducción progresiva de la distancia euclídea desde 1,60 m hasta alcanzar la tolerancia de llegada (0,22 m) en 75,07 s.

El registro de la terminal con las trazas de estado y tiempos de la misión se recoge en el Listado 1.

Listing 1: Registro de ejecución y retroalimentación asíncrona en consola (`nav2_send_goal.py`).

```
developer@santvz6:~/ros2_ws$ python3 p1_scripts/nav2_send_goal.py --x 1.5 --y -0.5 --
    yaw 90
[INFO] [nav2_goal_sender_node]: Inicializando cliente de navegacion autonoma Nav2 (
    use_sim_time=True)...
[INFO] [nav2_goal_sender_node]: Conectando con el servidor de accion "
    navigate_to_pose"...
[INFO] [nav2_goal_sender_node]: Enviando objetivo -> X: 1.50 m, Y: -0.50 m, Yaw: 90.0
     deg (1.57 rad) en frame "map"
[INFO] [nav2_goal_sender_node]: Objetivo ACEPTADO por Nav2. Iniciando ejecucion de
    trayectoria...
[Nav2 Feedback] Distancia restante: 1.60 m | Tiempo:  0.0 s | Recuperaciones: 0
[Nav2 Feedback] Distancia restante: 1.42 m | Tiempo:  7.3 s | Recuperaciones: 0
[Nav2 Feedback] Distancia restante: 1.17 m | Tiempo: 21.0 s | Recuperaciones: 3
[Nav2 Feedback] Distancia restante: 0.86 m | Tiempo: 46.7 s | Recuperaciones: 5
[Nav2 Feedback] Distancia restante: 0.22 m | Tiempo: 51.7 s | Recuperaciones: 5
[INFO] [nav2_goal_sender_node]: EXITO: El TurtleBot 4 ha alcanzado la pose objetivo
    en 75.07 s.
```

3 Bloque 4 – Manipulación Robótica (UR3e y MoveIt 2)

3.1 Contexto y Arquitectura de los Tres Pilares

El sistema desplegado integra el cobot de 6 grados de libertad **Universal Robots UR3e** en simulación, sustentado sobre tres componentes tecnológicos que interactúan de forma desacoplada:

1. **Gazebo Harmonic (`gz-sim`):** Motor de física de cuerpos rígidos (*DART Physics Engine*). Modela la gravedad, inercias de eslabones, restricciones de articulación (*revolute joints*) y dinámica de contactos.

2. **ros2_control (`controller_manager`):** Capa de abstracción de hardware en tiempo real. Gestiona el ciclo de vida del `joint_state_broadcaster` (lectura de encoders) y del `scaled_joint_trajectory_controller` (seguimiento de trayectorias articulares con escalado dinámico de velocidad ante sobrecargas).

3. **MoveIt 2 (`move_group`):** Núcleo de computación cinemática y planificación espacial. Gestiona el estado de la escena (*Planning Scene*), resuelve la cinemática inversa analítica/numérica y utiliza algoritmos de muestreo probabilístico (**OMPL**) para trazar caminos libres de colisiones.

3.2 Despliegue y Control Interactivo en RViz2

El ecosistema completo se inicializa mediante un único script de *launch* optimizado:

```
ros2 launch ur_simulation_gz ur_sim_moveit.launch.py ur_type:=ur3e
```

Este comando arranca simultáneamente el simulador Gazebo Harmonic y la interfaz de visualización 3D **RViz2** configurada con el plugin *MotionPlanning* y el árbol URDF del robot.

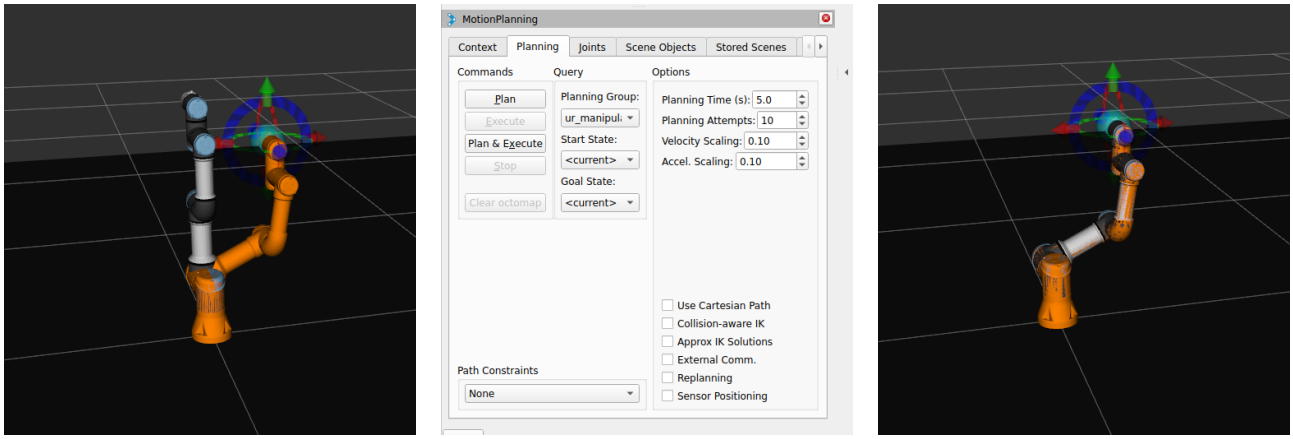
3.2.1 Operación en el Espacio de Trabajo

El control interactivo se realiza gobernando el efector final (`tool0`):

1. **Marcador Interactivo:** RViz dibuja un manipulador esférico con anillos ortogonales de rotación y flechas de traslación cartesianas (X, Y, Z) sobre el extremo del robot.

2. **Estado Objetivo (*Goal State*):** Al arrastrar el marcador con el ratón, se calcula en tiempo real la cinemática inversa y se proyecta la postura destino mediante un robot semitransparente de color naranja.

3. **Planificación y Ejecución (*Plan and Execute*):** En el panel *MotionPlanning*, la pestaña *Planning* ejecuta el planificador OMPL (algoritmo RRTConnect) y transmite los splines interpolados a `ros2_control`, haciendo que el brazo físico en Gazebo ejecute la trayectoria de forma suave.



(a) Arrastre interactivo (*Goal State*). (b) Panel *MotionPlanning*. (c) Trayectoria ejecutada con éxito.

Figura 5: Secuencia de interacción manual en RViz2: fijación de meta cartesiana 3D mediante marcador interactivo, cálculo de trayectoria OMPL y ejecución dinámica en Gazebo.

3.3 Introspección del Middleware ROS 2 (Sección 6.1)

Conforme a los requerimientos de la Sección 6.1 del guion, se procedió a interrogar al middleware utilizando herramientas CLI de introspección activa. Se detectaron **19 nodos funcionales** coordinando la simulación.

3.3.1 Mapeo Detallado de Nodos e Interfaces

La Tabla 2 resume las entradas, salidas, servicios y acciones de los nodos medulares del pipeline.

Nodo Activo	Tópicos Suscritos	Tópicos Publicados	Servicios / Acciones
`/move_group`	`/joint_states`, `/clock`, `/trajectory_execution_event`	`/display_planned_path`, `/monitored_planning_scene`	Acciones: `/move_action`, `/execute_trajectory`. Srv: `/compute_cartesian_path`
`/controller_manager`	`/clock`, `/robot_description`	`/controller_manager/activity`, métricas de lazo	Srv: `/switch_controller`, `/load_controller`
`/joint_state_broadcaster`	`/clock`	`/joint_states`, `/dynamic_joint_states`	Srv: Parámetros del broadcaster
`/scaled_joint_trajectory_controller`	`/clock`, consignas de trayectoria	`/controller_state`, eventos de transición	Acción: `/scaled_joint_trajectory_controller/follow_joint_trajectory`
`/robot_state_publisher`	`/joint_states`, `/clock`	`/tf`, `/tf_static`, `/robot_description`	Srv: Introspección URDF
`/rviz2_moveit`	`/display_planned_path`, `/tf`, `/tf_static`	Feedback de marcadores, `/goal_pose`	Cliente de acción: `/move_action`

Tabla 2: Mapeo de interfaces de los componentes principales del Bloque 4.

3.3.2 Grafo Computacional Dinámico (`rqt_graph`)

El análisis gráfico ejecutado con `ros2 run rqt_graph rqt_graph` (Figura 6) confirma la arquitectura desacoplada: `joint_state_broadcaster` actúa como el puente sensorial alimentando a `robot_state_publisher` y a `move_group`, mientras que la ejecución de comandos se delega al controlador articular de `ros2_control`.

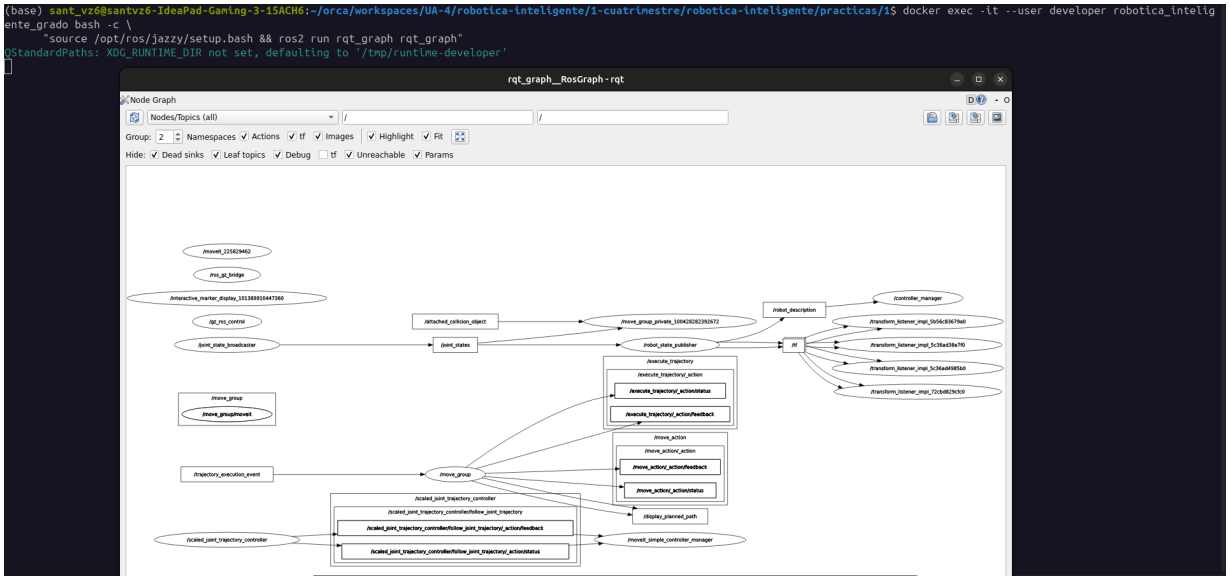


Figura 6: Grafo dinámico de nodos y tópicos del sistema capturado en vivo mediante `rqt_graph`.

3.4 Explicación del Flujo de Información y Cinemática

Para comprender cómo funciona el sistema en conjunto, analizamos el recorrido de la información desde que solicitamos una postura hasta que el robot se mueve en el simulador:

1. **Definición de la meta (Pose objetivo):**
   Se indica la posición (X, Y, Z) y la orientación deseada para el extremo del robot (`tool0`). Esta orden puede enviarse de forma manual arrastrando el marcador 3D en RViz2 o mediante comandos desde la terminal utilizando el script en Python.

2. **Cálculo de cinemática inversa y planificación (MoveIt 2):**
   El nodo `move_group` procesa la petición realizando dos pasos fundamentales:

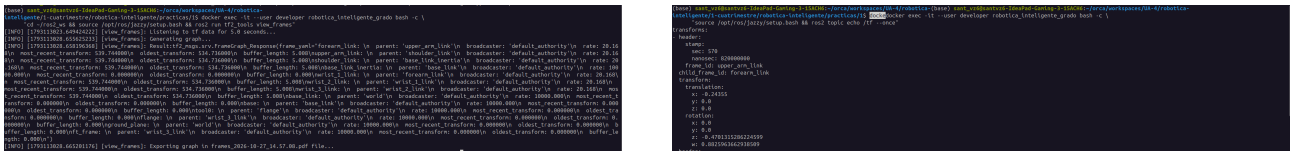
   - **Cinemática inversa (IK):** Calcula los ángulos necesarios en cada una de las 6 articulaciones del brazo $(\theta_1, \ldots, \theta_6)$ para situar el extremo en la coordenada espacial solicitada.
   - **Planificación de trayectoria:** Con los ángulos objetivo, el planificador (OMPL) busca un camino libre de colisiones entre la postura actual y la meta, generando una secuencia de puntos suave que respeta los límites de velocidad y aceleración del brazo.

3. **Ejecución del movimiento (`ros2_control`):**
   Una vez calculada la trayectoria, MoveIt la transmite al controlador articular de trayectoria mediante la acción de ROS 2 `FollowJointTrajectory`. El nodo `scaled_joint_trajectory_controller` guía a los motores para recorrer la ruta en el tiempo previsto.

4. **Simulación física y realimentación sensorial (Gazebo y TF2):**
   Gazebo aplica el movimiento a las articulaciones físicas del robot en el entorno virtual. En paralelo, los sensores del simulador leen la posición real de cada articulación y la publican en el tópico `/joint_states`. A partir de estas lecturas, el nodo `robot_state_publisher` actualiza el árbol de transformaciones geométricas (`/tf`), permitiendo que RViz2 refleje en tiempo real la postura exacta del robot.

3.5 Análisis e Interpretación del Árbol de Transformaciones (TF2)

La comprensión espacial de la cadena cinemática se verificó mediante la herramienta `ros2 run tf2_tools view_frames` (Figura 7).



(a) Extracción de *frames* con `view_frames`. (b) Inspección en vivo con `echo /tf --once`.

Figura 7: Introspección del árbol espacial de transformaciones geométricas (TF2) en consola.

El árbol jerárquico se compone de **14 marcos de referencia** encadenados:

$$\text{world} \longrightarrow \text{base_link} \longrightarrow \text{base_link_inertia} \longrightarrow \text{shoulder_link} \longrightarrow \text{upper_arm_link}$$
$$\longrightarrow \text{forearm_link} \longrightarrow \text{wrist_1_link} \longrightarrow \text{wrist_2_link} \longrightarrow \text{wrist_3_link} \longrightarrow \text{flange} \longrightarrow \text{tool0}$$

- **Transformaciones Estáticas (`/tf_static`):** Invariantes en el tiempo, tales como la fijación del robot al suelo (`world ->base_link`), la inercia interna de la peana y la distancia fija entre la brida mecánica y el centro de herramienta (`flange ->tool0`).
- **Transformaciones Dinámicas (`/tf`):** Publicadas a una frecuencia nominal de **20.16 Hz** por `robot_state_publisher`. Cada enlace móvil transmite un vector de traslación constante (longitud del eslabón) y un cuaternión de rotación unitario variable que refleja en tiempo real el ángulo del servomotor.

3.6 Tarea de Programación Opcional: Control en Python (Sección 6.2)

Para aspirar a la máxima calificación según la Sección 6.2 del guion, se programó y validó el script en Python `ur3e_send_pose.py` en el espacio de trabajo persistente (`~/ros2_ws/p1_scripts/`). Este script prescinde enteramente de la interfaz gráfica RViz2, conectándose como cliente directo a las acciones de ROS 2.

Se validaron tres modos de control en la consola (Figura 8):

1. **Prueba A – Modo Articular Directo (`--mode joint`):**
   Envío directo de la configuración articular $\boldsymbol{\theta} = [0{,}0, -1{,}57, 1{,}57, -1{,}57, -1{,}57, 0{,}0]$ rad al nodo `scaled_joint_trajectory_controller`. El manipulador flexiona el codo en ángulo recto con perfil suave interpolado en 4.0 segundos.

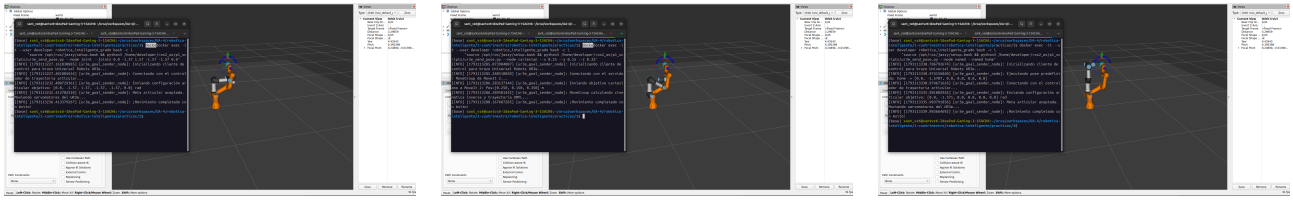
   ```
   python3 ur3e_send_pose.py --mode joint --joints 0.0 -1.57 1.57 -1.57 -1.57 0.0
   ```

2. **Prueba B – Modo Cinemático Cartesiano 3D (`--mode cartesian`):** Fijación de coordenada espacial ($X = 0{,}25\,\text{m}, Y = 0{,}15\,\text{m}, Z = 0{,}35\,\text{m}$) con orientación vertical (pitch $= \pi$). La orden viaja al servidor de acción `/move_action` de MoveIt 2, el cual calcula cinemática inversa y trayectoria OMPL sin colisiones antes de ejecutar.

   ```
   python3 ur3e_send_pose.py --mode cartesian --x 0.25 --y 0.15 --z 0.35
   ```

3. **Prueba C – Poses Semánticas SRDF (`--mode named`):** Carga de posturas preestablecidas de fábrica (`home`, `up`, `test_configuration`) definidas en el archivo semántico SRDF de Universal Robots.

   ```
   python3 ur3e_send_pose.py --mode named --named home
   ```



(a) Prueba A: Control Articular. (b) Prueba B: Meta Cartesiana 3D. (c) Prueba C: Pose Semántica *home*.

Figura 8: Validación experimental de la tarea de programación opcional (Sección 6.2): ejecución de comandos desde la terminal y respuesta síncrona del robot en RViz2 y Gazebo.